

Test Case Template

To complete the test case plan, fill out the information for Project, Course, Description and Date in the header below.

Project: Todo Application Test Plan	
Course: CSD 380 – DevOps and Software Testing	
Description: This document explains the functional test plan for the Todo web app. It includes five test cases that check whether users can add, see, update, and delete tasks, and whether a custom 404 page appears for invalid URLs. Each test makes sure the app works the way it's supposed to.	
Date: 2025/11/13	

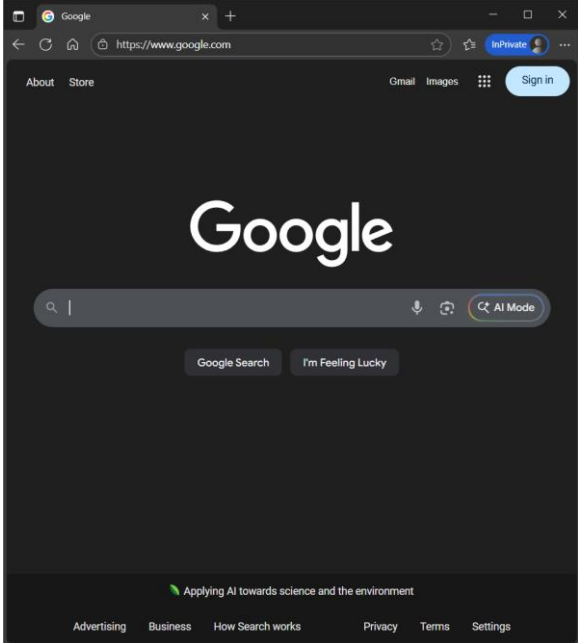
TABLE OF CONTENTS

Introduction	1
Test Objectives	2
Test Cases	3
Test 1 – View Todo List	3
Test 2 – Create a Todo Item	4
Test 3 – Edit an Existing Todo Item	4
Test 4 – Delete a Todo Item	5
Test 5 – Handle Invalid URLs	6

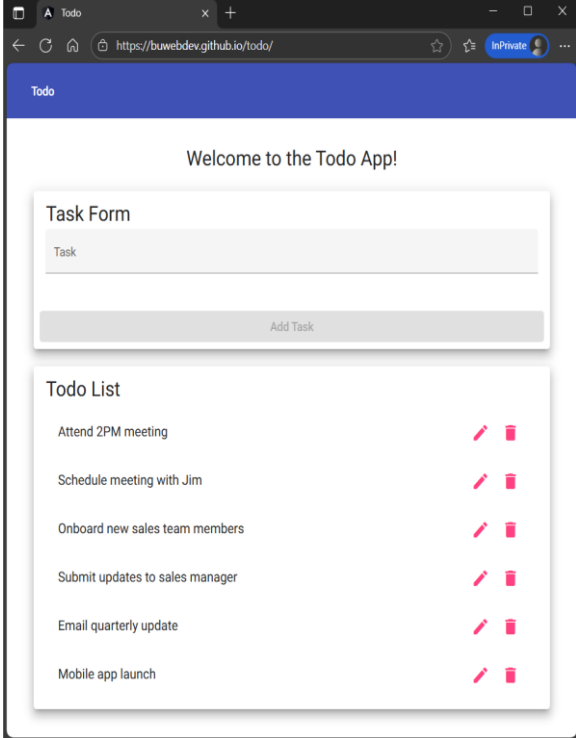
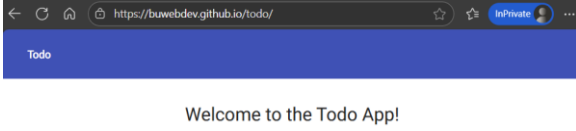
Test Case Template

For each test, the developer should: provide a test description, a test objective, the developer name and date tested. For each step, fill out the actions to be taken and describe the expected results, and indicate yes or no for each step depending upon the result. The peer tester should provide their name, date tested, indicate yes or no for each step depending upon the result, and a screenshot (thumbnail) of the result.

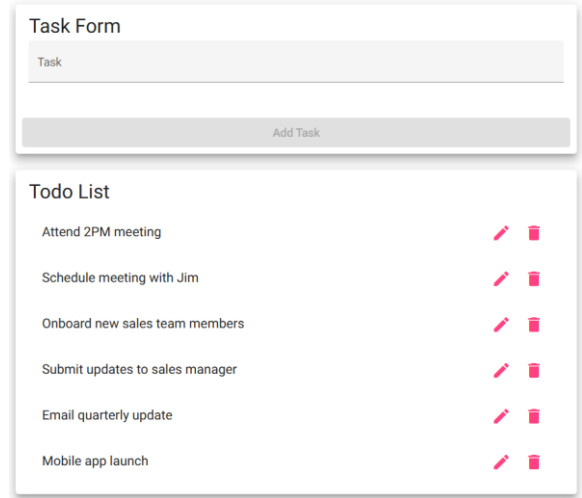
Test Case Template

Test 1	Access the Todo Application			
	Test Objective: Verify that users can access the Todo application through the provided URL.	Developer: Jose Flores Date tested: 2025/11/13	Peer tester: Clint Scott Date tested: 2025/11/16	
Step	Action	Expected results:	Developer pass/fail	Tester pass/fail + Screenshot
1	Open a web browser.	Browser opens.	PASS	PASS 

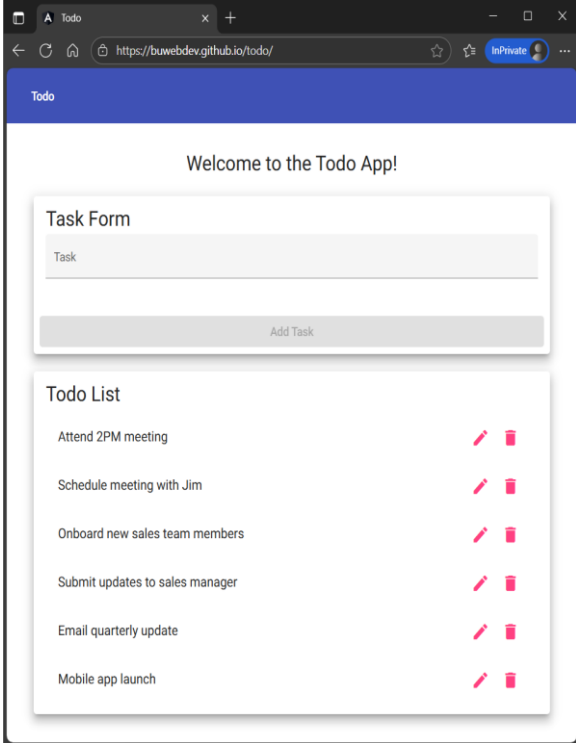
Test Case Template

2	Go to https://buwebdev.github.io/todo/ .	The Todo homepage loads.	PASS	PASS 
3	Verify that “Welcome to the Todo App!” is displayed.	Welcome message appears clearly at the top.	PASS	PASS 

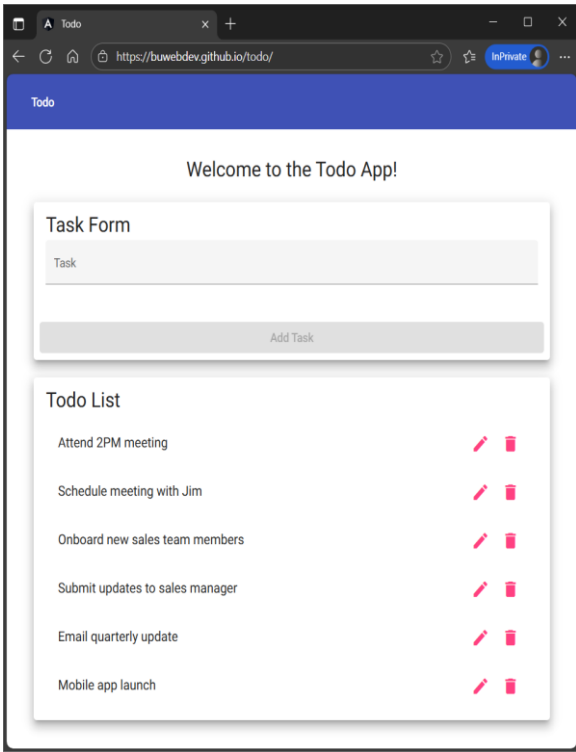
Test Case Template

4	Check "Task Form" and "Todo List" sections appear.	Both sections are visible with sample tasks.	PASS	PASS 
---	--	--	------	--

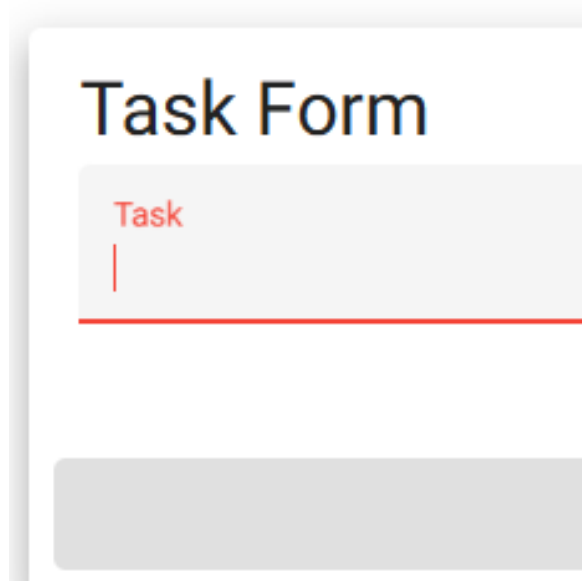
Test Case Template

5	Refresh the page.	Page reloads with no errors.	PASS	PASS 
Comments	All steps in this test case have passed, successfully confirming that the Todo Application can be accessed. The main components, including the 'Welcome' message, 'Task Form,' and 'Todo List,' are displayed correctly upon both loading and refreshing the page. The initial load time was quick, and the page's content rendered without any noticeable issues or visual defects. This verifies that the application's basic view functionality meets the required standards.			

Test Case Template

Test 2	Create a New Todo Item			
	Test Objective: Verify that a user can successfully add a new task to the list.	Developer: Jose Flores Date tested: 2025/11/13	Peer tester: Clint Scott Date tested: 2025/11/16	
Step	Action	Expected results:	Developer pass/fail	Tester pass/fail
1	Open the Todo app in a browser.	App loads.	PASS	PASS 

Test Case Template

2	Click in the Task input box.	Cursor appears, ready for typing.	PASS	PASS 
---	------------------------------	-----------------------------------	------	---

Test Case Template

3	Type "CSD 380 Assignment."	Text displays correctly in the field.	PASS	PASS Task Form Task CSD 380 Assignment.
---	----------------------------	---------------------------------------	------	--

Test Case Template

4	Click the Add Task button.	The new task appears at the bottom of the Todo List.	PASS	PASS Todo List - Attend 2PM meeting - Schedule meeting with Jim - Onboard new sales team members - Submit updates to sales manager - Email quarterly update - Mobile app launch - CSD 380 Assignment.
---	----------------------------	--	------	---

Test Case Template

5	Check the new item.	Task remains visible after action completes.	PASS	PASS Todo List - Attend 2PM meeting - Schedule meeting with Jim - Onboard new sales team members - Submit updates to sales manager - Email quarterly update - Mobile app launch - CSD 380 Assignment.
Comments	The core functionality for creating a new Todo item works as expected. The new task, "CSD 380 Assignment," appears instantly in the list after clicking "Add Task." However, the test plan could be more efficient, as Steps 4 and 5 are nearly identical, both confirming the task's visibility immediately after it is created. Replace Step 5 with a different action, such as verifying that the input field is automatically cleared after submission. This change would provide more comprehensive coverage of the "Create" process.			

Test Case Template

Test 3	Edit an Existing Todo Item			
	Test Objective: Verify that a user can modify an existing task.	Developer: Jose Flores Date tested: 2025/11/13	Peer tester: Clint Scott Date tested: 2025/11/16	
Step	Action	Expected results:	Developer pass/fail	Tester pass/fail
1	Make sure a task is in the list	Task is visible	PASS	PASS Todo List - Attend 2PM meeting - Schedule meeting with Jim - Onboard new sales team members - Submit updates to sales manager - Email quarterly update - Mobile app launch - CSD 380 Assignment.

Test Case Template

2	Click the Edit button next to the task.	Task becomes editable.	PASS	PASS Edit Form Task Attend 2PM meeting Save
3	Change the task text to "CSD 380 Assignments to CSD 380 Projects."	Text updates in the input area.	PASS	PASS Edit Form Task CSD 380 Projects.

Test Case Template

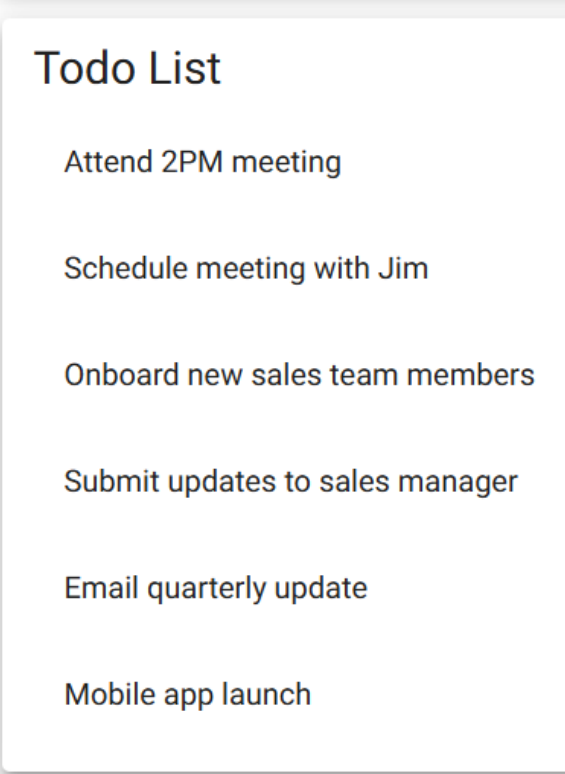
4	Save the edit.	Task text updates in the list.	PASS	PASS Todo List Attend 2PM meeting Schedule meeting with Jim Onboard new sales team members Submit updates to sales manager Email quarterly update Mobile app launch CSD 380 Projects.
---	----------------	--------------------------------	------	--

Test Case Template

5	Refresh the page.	The edited task goes back to the original text because changes aren't saved.	FAIL	FAIL Todo List Attend 2PM meeting Schedule meeting with Jim Onboard new sales team members Submit updates to sales manager Email quarterly update Mobile app launch
Comments	The ability to edit an existing task worked correctly while on the page, with the text updating immediately after saving (Steps 1-4 were successful). However, the test failed on Step 5 because refreshing the page reverted the edited task to its original state, indicating that the changes are not being saved. Additionally, when editing, the user interface lacks a 'Cancel' option to discard changes. Adding a 'Cancel' function alongside the 'Save' option would enhance the user experience.			

Test 4	Delete a Todo Item
--------	---------------------------

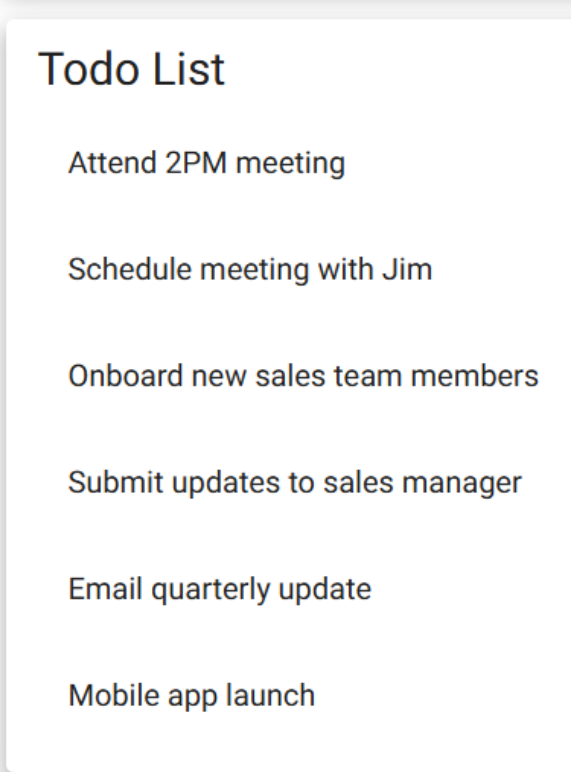
Test Case Template

	Test Objective: Confirm that users can delete a task from the list.	Developer: Jose Flores Date tested: 2025/11/13	Peer tester: Clint Scott Date tested: 2025/11/16	
Step	Action	Expected results:	Developer pass/fail	Tester pass/fail
1	Find a task in the list (ex., "Mobile app launch").	Task is visible.	PASS	PASS 

Test Case Template

2	Click the Trash icon.	The selected task disappears from the list.	PASS	PASS Todo List Attend 2PM meeting Schedule meeting with Jim Onboard new sales team members Submit updates to sales manager Email quarterly update
---	------------------------------	---	------	--

Test Case Template

3	Refresh the page.	The deleted task comes back because the app doesn't save changes.	FAIL	FAIL 
---	-------------------	---	------	--

Test Case Template

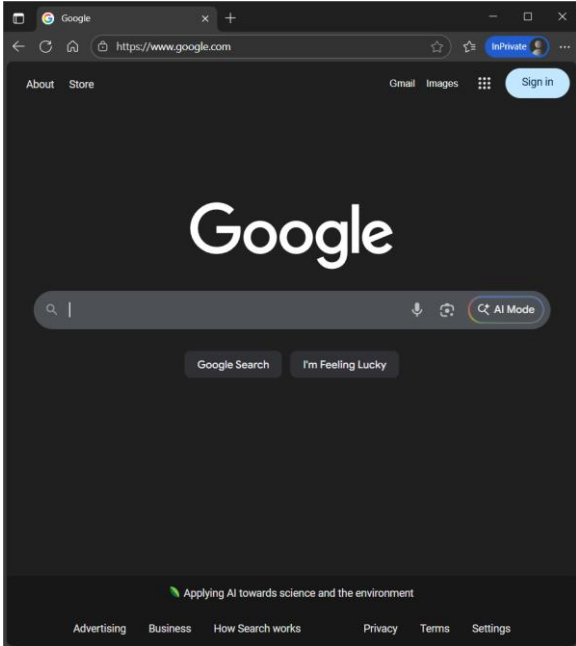
4	Try deleting another task.	Function repeats successfully.	PASS	PASS Todo List Attend 2PM meeting Schedule meeting with Jim Onboard new sales team members Email quarterly update Mobile app launch
---	----------------------------	--------------------------------	------	--

Test Case Template

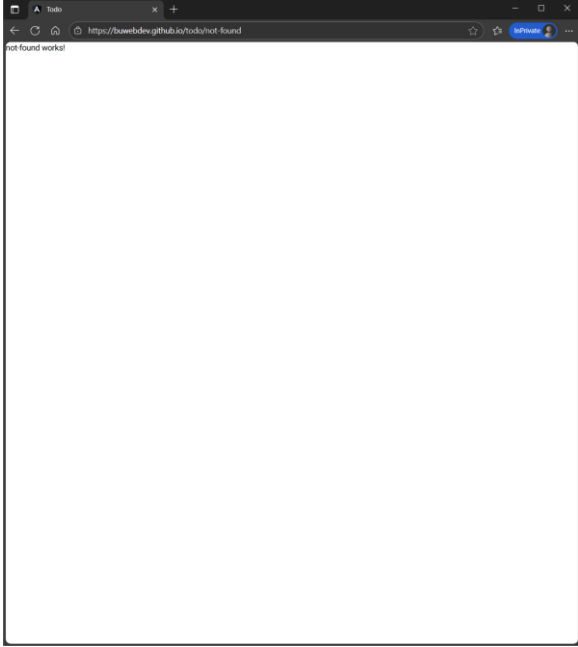
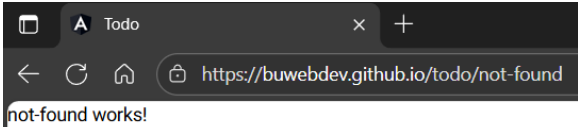
5	Confirm list updates immediately.	Remaining items stay visible.	PASS	PASS Todo List - Attend 2PM meeting - Schedule meeting with Jim - Onboard new sales team members - Email quarterly update - Mobile app launch
Comments	The immediate function for deleting a task works successfully, as Steps 1, 2, 4, and 5 have passed. The item disappears from the list instantly when the Trash icon is clicked. However, the test failed on Step 3 because, similar to the edit functionality, refreshing the page causes the deleted task to reappear. The ongoing issue of data loss upon page reload needs to be addressed for both the delete and edit features to meet user expectations for a functional Todo application.			

Test 5	Handle Invalid URLs			
	Test Objective: Ensure the app displays a custom 404 error page when visiting an invalid URL.	Developer: Jose Flores Date tested: 2025/11/13	Peer tester: Clint Scott Date tested: 2025/11/16	

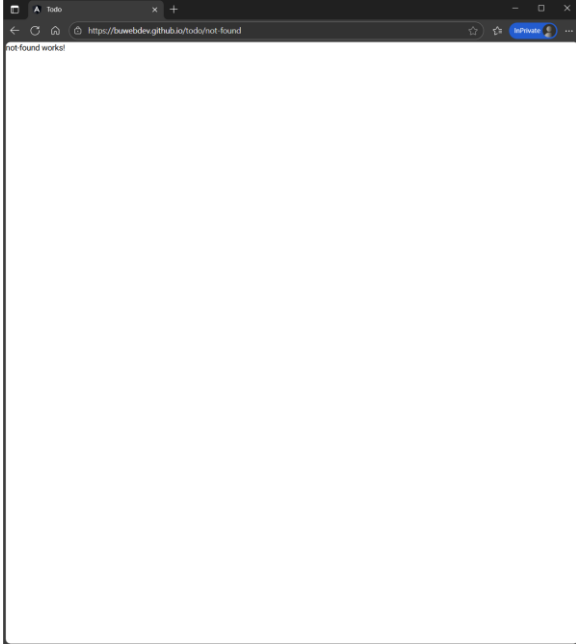
Test Case Template

Step	Action	Expected results:	Developer pass/fail	Tester pass/fail
1	Open a browser.	Browser launches successfully.	PASS	PASS 

Test Case Template

2	Navigate to https://buwebdev.github.io/todo/fakepage .	Page loads a “not-found works!” message.	PASS	PASS  A screenshot of a web browser window. The address bar shows the URL 'https://buwebdev.github.io/todo/not-found'. The page content is a plain white background with the text 'not found works!' at the top left.
3	Check if the message.	Displays app's own 404 page with message: “not-found works!”	PASS	PASS  A screenshot of a web browser window. The address bar shows the URL 'https://buwebdev.github.io/todo/not-found'. The page content is a plain white background with the text 'not found works!' at the top left.

Test Case Template

4	Return to main URL.	App homepage loads normally.	PASS	FAIL  A screenshot of a web browser window. The address bar shows the URL 'https://baweibdev.github.io/todo/not-found'. The page content is a plain white background with the text 'not found work!' at the top left. The browser's interface, including the address bar, back/forward buttons, and window controls, is visible at the top.
---	---------------------	------------------------------	------	--

Test Case Template

5	Refresh the main page.	Home displays without error.	PASS	FAIL 
Comments	The application successfully achieves its objective by displaying a custom "not found" 404 page for invalid URLs. This page maintains brand consistency and is an improvement over the default browser error message. However, Steps 4 and 5 fell short due to the lack of navigation options on the error page; there is currently no link or button to return to the homepage. To enhance user experience and accessibility, a clear hyperlink or button should be added to the 404 page, allowing users to navigate back to the functional homepage easily.			